

PROJECT REFERENCE / IMPLEMENTED SYSTEM

News Corroboration Engine

Libraries, database and news sources

An explanation of what the original design proposed, what the first Python version used, and what the later news-search update added.

Prepared from the project code and installed package metadata on **22 September 2026**. Historical integration and test results are dated separately; this report does not claim that publisher availability was rechecked today.

Environment	Verified project setup
Language	Python 3.12.14 in the project environment; package requires Python 3.11 or newer.
Application	FastAPI backend, Uvicorn server, plain HTML/CSS/JavaScript dashboard.
Persistence	One local SQLite database; SQLite library 3.53.1 . Default file: data/news.db .
News inputs	Six configured publisher integrations, plus on-demand Google News RSS search.

How the project evolved

Stage	What was used or added
Original PDF design	Proposed a Python pipeline with RSS, NLP, matching, FastAPI, live alerts, SQLite for development and possible production technologies.
First Python version	Implemented feed ingestion, keyword flags, TF-IDF claim matching, SQLite storage/cache, API, WebSocket updates and a dashboard.
Search update	Added names and short abbreviations, configurable aliases, Google News lookup, date filters and a crime-reporting filter. The existing backend, database and core libraries stayed in place.

Two different jobs

Claim checking compares a statement with locally indexed reporting and returns a heuristic verdict. **News search** finds mentions and related coverage; it does not decide whether a person is guilty or a claim is true.

01 / APPLICATION LIBRARIES

What the Python libraries do

These packages are used by the implemented application. Versions below were read from the project's installed environment and match its recorded dependency snapshot.

Library / version	Purpose in this project
FastAPI 0.141.1	Defines REST routes for searches, claim checks, sources, articles and history; provides the WebSocket endpoint and serves dashboard files.
Uvicorn 0.53.0	Runs the FastAPI application as an ASGI web server. The development launcher binds to 127.0.0.1:8000.
HTTPX 0.28.1	Makes asynchronous HTTP requests to publisher sites and Google News. Used with timeouts, size bounds and redirect checks.
feedparser 6.0.14	Parses RSS/Atom entries into titles, URLs, descriptions and publication metadata, including the Google News search feed.
Beautiful Soup beautifulsoup4 4.15.0	Removes HTML/script markup, discovers feed links and extracts article text for the HTML fallback. Imported as bs4.
scikit-learn 1.9.1	Weights words and character sequences with TF-IDF (term frequency-inverse document frequency), then measures cosine similarity to rank claim matches.
YAKE 0.7.3	Extracts up to eight English keyphrases, with phrases up to two words. Urdu text instead uses normalized word frequencies.
python-dotenv 1.2.3	Loads local .env settings for the database path, polling, optional models, API key and other configuration.
NumPy 2.5.3	Handles arrays, score combinations and ranked indices in the matching engine.
Pydantic 2.13.5	Validates request fields, length limits, search mode, date range and result limits before executing a request.
Starlette 1.6.0	Supplies FastAPI's web infrastructure, static-file support and thread-pool helper for blocking work.

Dependency declaration: the first eight packages are explicitly listed under project dependencies in pyproject.toml. NumPy, Pydantic and Starlette are imported directly by project code but currently arrive through those dependencies. requirements-tested.txt records exact installed versions.

Terms in plain language: RSS/Atom are machine-readable news feeds. An API lets the dashboard request data from the backend. ASGI is the interface connecting an asynchronous Python web server to its application. WebSocket keeps a connection open so new articles can be pushed to the browser.

02 / STORAGE

One database, five tables

The project uses **SQLite** through Python's built-in **sqlite3** module. It does not require a separate database server. SQL statements are written directly in `storage.py`; no ORM such as SQLAlchemy is used.

Table	What it stores and why
articles	Canonical URL and hash ID, publisher, title, short snippet, publication/collection/update times, severity and JSON metadata. Supports retrieval, deduplication and article updates.
history	Timestamped query results, including submitted text, matches and claim verdicts or search labels. Used by the Recent checks view.
cache	A hashed lookup key, expiry time and JSON result. Reuses repeated claim checks and searches without a Redis service.
source_status	The last recorded ingestion status for each configured publisher, including errors or HTTP status when available.
metadata	An article revision counter, increased on changed article writes. Query caches are invalidated when articles change.

Storage rules and defaults

Snippets, not full articles: ingestion may examine longer content in memory for keyword scanning, but stores a title of up to 400 characters and a snippet of 500 characters by default (configuration maximum: 1,000). Links, extracted terms and flags remain attached.

Cache lifetime: normally 120 seconds. A failed live-search result is cached for 10 seconds. Expired entries are removed during cache writes; article changes clear the cache.

History: saving is enabled by default, with 30-day retention. Old history is filtered on reads and purged on subsequent recorded queries. Turning saving off does not delete old records.

Concurrency: SQLite uses write-ahead logging (WAL), transactions, a busy timeout and a thread lock. The application is designed for a single server process; multiple workers would each start their own ingestion loop and live-client list.

Where live-search results go

Google News results are **not inserted into the articles table** and do not become supporting publishers for claim verdicts. Their bounded snippets can still appear in **cache** and, when enabled, **history**. TF-IDF matrices and optional embeddings are held in memory, not in a separate vector database.

03 / NEWS INPUTS

Which sources are used?

Six publisher integrations remain configured and enabled in `news_engine/config/sources.json`. They are the same core publisher set as in the first implementation. The listed outcomes come from the recorded **18 September 2026** ingestion test, not a fresh availability check.

Publisher / configured endpoint	Recorded outcome
DAWN https://www.dawn.com/feeds/home	RSS succeeded; 24 articles changed.
ARY News https://arynews.tv/feed/	RSS succeeded; 30 articles changed.
Geo News https://www.geo.tv/rss/1/1	RSS succeeded; 20 articles changed.
Hum News https://humnews.pk/feed/	HTTP 403 from publisher; no articles added.
Dunya News https://dunyanews.tv/en/rss	Feed URL returned HTML; HTML fallback succeeded with 12 articles.
Samaa https://www.samaa.tv/feed/	Connection failed; no articles added.

How publisher ingestion works

HTTPX fetches a configured feed and feedparser reads its entries. When suitable, BeautifulSoup discovers a feed or extracts links/text from publisher pages. Scraping checks robots.txt; allowed publisher hosts, public IP checks and bounded responses limit outbound fetching.

A cycle takes up to 30 feed entries or 12 HTML fallback links per publisher, then waits 120 seconds before the next completed-cycle run. New or changed articles are stored and pushed to connected browsers. This is periodic collection, not an exhaustive or instantaneous news archive.

Added in the search update: Google News

Endpoint: <https://news.google.com/rss/search>

The query is sent with related terms, a date window and Pakistan/English edition settings (hl=en-PK, gl=PK, ceid=PK:en). Up to 100 returned feed entries are examined.

Google News is a **discovery/aggregation source**, not the author of every article. Each result displays the publisher reported by the feed and links through Google News. Results can come from publishers beyond the six fixed integrations, including international outlets.

Live search is enabled by default, requires internet access and currently needs no API key in this implementation. The user can disable it. If it fails, the response says so and retains local-index results. The public RSS route has no availability guarantee in this project.

04 / PROCESSING & DESIGN CHOICES

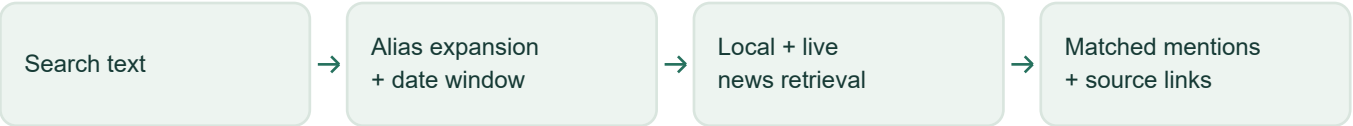
What happens to the text?

Full-claim checking



Word/bigram similarity contributes 75% and character-ngram similarity 25% to the default lexical score. The heuristic checks overlapping details, numbers and negation; at least two distinct publishers with different normalized headlines are required for Corroborated. Claim retrieval uses a 7-day window and at most 5,000 indexed articles by default.

Name, topic and abbreviation search



Whole-word matching and search_aliases.json support terms such as CTD, DoS, DDoS and cyber crime without the long-claim score threshold. All query concepts must match. CTD-only matches need a police context; DoS-only matches need a computing context. Search defaults to 30 days; the dashboard also offers 7 days and 1 year. These are filters over available coverage, not promises of a complete archive.

Optional or proposed: not part of the default runtime

Technology	Actual status / replacement
sentence-transformers	Optional integration; not installed. Can add multilingual sentence embeddings after explicit installation and enablement.
spaCy	Optional integration; not installed. Default named-entity output is explicitly labeled rule-based candidates.
PostgreSQL / Redis	Proposed production choices only. No adapters are implemented; SQLite handles data and result caching.
APScheduler	Proposed scheduler; not used. Python asyncio starts and repeats the ingestion task within the API lifecycle.
Scrapy / aiohttp / RAKE	Not used. The selected alternatives are Beautiful Soup, HTTPX and YAKE respectively.
React	Proposed optional frontend; not used. The dashboard is plain HTML/CSS/JavaScript served by FastAPI.

The optional encoder is sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. When enabled, embeddings contribute 45% and lexical scores 55%. A separate spaCy model must also be installed to use trained NER. No trained Urdu NER model, hosted LLM API, paid news API or external vector database is used by default.

05 / SUPPORT & EVIDENCE

Supporting tools and references

Built-in Python components

asyncio handles scheduling, bounded concurrency and live-client queues. **sqlite3** provides database access; **threading** protects shared operations. **re**, **unicodedata** and **collections.Counter** clean and match text. **hashlib** creates stable URL/cache hashes; **json** serializes payloads and configuration. **datetime**, **calendar** and **email.utils** normalize dates. **urllib.parse** builds/checks URLs; **urllib.robotparser** reads robots rules; **socket** and **ipaddress** check destinations. **secrets** compares API keys. **argparse**, **pathlib**, **dataclasses**, **os**, **typing**, **contextlib** and **logging** support the CLI, settings and lifecycle.

Development tool	Role / verified version
pytest / pytest-asyncio	Synchronous and asynchronous tests; 9.1.1 / 1.4.0. The last recorded suite passed 81 tests on 18 September.
Ruff	Python linting/formatting; 0.16.8. Existing validation records report passing checks.
setuptools / pip / venv	Package building, dependency installation and environment isolation. setuptools >=75 is the declared build requirement.
Node.js / browser	Node.js checked dashboard JavaScript syntax; browser checks verified the UI. Neither requires a Node-based production frontend server.

Indirect packages in the recorded environment

Networking/server: anyio, httpcore, h11, httptools, websockets, watchfiles, certifi, idna, click, colorama, PyYAML. **Parsing/keyphrases:** feedparser-sgmllib, soupsieve, jellyfish, networkx, segtok, regex, tabulate. **Numerical/model support:** scipy, joblib, threadpoolctl, narwhals, cloudpickle. **Validation/typing:** pydantic_core, annotated-types, annotated-doc, typing-inspection, typing_extensions. **Test/tool support:** iniconfig, pluggy, Pygments, packaging. Exact pins are in requirements-tested.txt; not every installed helper is directly imported by application code.

Evidence used for this report

Original design: News_Corroboration_Engine_Implementation (1).pdf, supplied by the user, especially its architecture and technology summary.
Dependencies: pyproject.toml; requirements-tested.txt; read-only checks of the project's installed package metadata, Python and SQLite versions.
Application code: news_engine/api.py, ingestion.py, matching.py, search.py, text.py, storage.py, live.py and settings.py; static/index.html and static/app.js.
Configuration: news_engine/config/sources.json, keywords.json, search_aliases.json and .env.example. Reported settings are code defaults, not a disclosure of private .env values.
Historical evidence: README.md and VALIDATION.md, including the 18 September test results and 86-article ingestion run. No new test-suite or publisher-availability run was needed to author this report.

Interpretation

A retrieval score is not a truth probability. A name match does not verify identity or guilt. Optional packages and proposed production technologies are clearly separated from what the code currently uses. ReportLab, pypdf and Poppler were used only to create and inspect **this PDF**; they are not application dependencies.